

Containers

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercícios

Laboratório Prático: Trabalhar com Docker Compose

Objetivo: Este laboratório irá guiá-lo através dos fundamentos da criação, gestão e implementação de aplicações usando o Docker Compose. Irá aplicar os conceitos de imagens, contentores, volumes e redes para construir e executar aplicações de serviço único e multi-serviço.

Pré-requisitos:

- Um computador com um navegador web moderno e um editor de texto.
 - Docker e Docker Compose instalados (consulte a secção de instalação abaixo).
-

Instalar o Docker no Debian

Se estiver a usar um anfitrião Linux, siga estes passos no seu terminal para instalar a versão mais recente do Docker (para a instalação manual pode seguir estas [instruções](#)).

Para simplificar o processo de instalação, pode executar o seguinte script bash (funciona em Ubuntu e Debian). Este irá configurar o repositório Docker, instalar o Docker Engine e configurar as permissões do seu utilizador:

```
./classes/class_03/02_support/docker_setup.sh
```

Dica: Se encontrar um erro de “permissão negada” (permission denied) ao executar comandos docker, certifique-se de que concluiu a instalação e terminou/iniciou a sessão novamente. Como solução rápida, pode colocar sudo antes dos comandos, mas configurar o grupo é a abordagem recomendada.

Exercício 1: “Hello, World” com Docker Compose

Objetivo: Compreender a estrutura básica de um ficheiro `compose.yml` e executar uma imagem pré-construída.

1. A partir do seu diretório de trabalho, crie uma nova pasta para este exercício:

```
mkdir ex01  
cd ex01
```

2. Dentro da pasta, crie um novo ficheiro chamado `compose.yml` com o seguinte conteúdo:

```
services:  
  hello:  
    image: hello-world
```

3. Execute a aplicação:

```
docker compose up
```

4. Observe o resultado. O contentor hello-world irá arrancar, imprimir a sua mensagem e depois terminar.
5. Limpe o contentor criado:
`docker compose down`

Verificação: Deverá ver a mensagem de boas-vindas do Docker que começa com "Hello from Docker!" na saída do terminal após o passo 3. Após o passo 5, executar `docker compose ps` não deverá mostrar nenhum contentor.

Dica: O ficheiro `compose.yml` (anteriormente chamado `docker-compose.yml`) é o nome de ficheiro padrão que o Docker Compose procura. Pode usar um nome diferente com a flag `-f`: `docker compose -f meu_ficheiro.yml up`.

Exercício 2: Construir uma Imagem de Servidor Web Personalizada

Objetivo: Usar um Dockerfile com o Docker Compose para criar uma imagem de aplicação autónoma.

1. A partir do seu diretório de trabalho, crie a estrutura de pastas:

```
mkdir -p ex02/my-website
cd ex02
```

2. Dentro de my-website, crie um ficheiro chamado `index.html`:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Esta pagina foi construida dentro da imagem Docker!</h1>
</body>
</html>
```

3. Na raiz da pasta ex02, crie um Dockerfile:

```
FROM nginx:alpine
COPY ./my-website /usr/share/nginx/html
```

4. Na mesma pasta, crie o seu ficheiro `compose.yml`:

```
services:
  webserver:
    build: .
    ports:
      - "8080:80"
```

5. Construa e inicie o serviço. A flag `-d` executa-o em segundo plano (modo detached):

```
docker compose up --build -d
```

6. Abra o seu navegador em `http://localhost:8080`. Deverá ver a sua página web personalizada.

Verificação:

- Execute `docker compose ps` para confirmar que o contentor está a correr e saudável.
- Execute `docker compose images` para ver a imagem que foi construída.
- Use `curl http://localhost:8080` como alternativa ao navegador.

Dica: A flag `--build` força o Compose a reconstruir a imagem antes de iniciar. Sem ela, o Compose reutiliza a imagem construída anteriormente. Use sempre `--build` após alterar o Dockerfile ou quaisquer ficheiros que sejam copiados para a imagem.

Limpeza: Quando terminar, pare e remova tudo com:

```
docker compose down
```

Exercício 3: Desenvolvimento em Tempo Real com Volumes

Objetivo: Compreender como os volumes (bind mounts) permitem alterar o conteúdo do seu site sem reconstruir a imagem.

1. A partir do seu diretório de trabalho, crie a estrutura de pastas:

```
mkdir -p ex03/my-website  
cd ex03
```

2. Crie um ficheiro my-website/index.html:

```
<!DOCTYPE html>  
<html>  
<body>  
  <h1>Ola a partir de um volume mount!</h1>  
</body>  
</html>
```

3. Crie um ficheiro compose.yml. Desta vez, usamos a imagem padrão nginx:alpine e montamos a nossa pasta local como um volume. **Não é necessário Dockerfile.**

```
services:  
  webserver:  
    image: nginx:alpine  
    ports:  
      - "8080:80"  
    volumes:  
      - ./my-website:/usr/share/nginx/html:ro
```

4. Inicie o serviço:

```
docker compose up -d
```

5. Abra o seu navegador em http://localhost:8080 para confirmar que está a funcionar.

6. **Atualização em Tempo Real:** Enquanto o contentor está a correr, **edite o ficheiro index.html** na sua máquina anfitriã. Altere o cabeçalho para <h1>Atualizacao em tempo real com um Volume!</h1>.

7. Guarde o ficheiro e **atualize o seu navegador**. A alteração aparece instantaneamente.

Verificação: Após editar o index.html, pode verificar a alteração a partir da linha de comandos:

```
curl http://localhost:8080
```

Dica: Repare na flag :ro (apenas de leitura) no final da montagem do volume. Esta é uma boa prática quando o contentor deve apenas ler ficheiros do anfitrião e nunca escrever neles. Evita que o contentor modifique acidentalmente os seus ficheiros de origem.

Conceito Chave – Build vs. Volume: Compare o Exercício 2 (build) com o Exercício 3 (volume). Construir cria uma imagem portátil e autónoma ideal para **produção**. Os volumes criam uma ligação em tempo real ideal para **desenvolvimento**. Compreender quando usar cada abordagem é fundamental.

Limpeza:

```
docker compose down
```

Exercício 4: Conteúdo Rico em Cache com Varnish e NGINX

Objetivo: Construir uma aplicação web de duas camadas com uma cache HTTP Varnish a servir uma página web rica a partir de um backend NGINX. Este exercício introduz ficheiros compose multi-serviço, dependências de serviço e redes internas.

1. A partir do seu diretório de trabalho, crie a Estrutura de Ficheiros:

```
mkdir -p ex04/my-dynamic-website  
cd ex04
```

2. Crie o Conteúdo Web:

- Encontre um GIF animado online e guarde-o dentro de my-dynamic-website como animation.gif. Por exemplo, pode usar este: [docker.gif](https://www.docker.com/images/dockerhub.gif).
- Dentro de my-dynamic-website, crie um ficheiro index.html para exibir o GIF:

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8">
  <title>Teste de Cache Varnish</title>
  <style> body { font-family: sans-serif; text-align: center; } </style>
</head>
<body>
  <h1>Esta pagina esta a ser colocada em cache pelo Varnish!</h1>
  
</body>
</html>
```

3. Crie a Configuração do Varnish:

- Dentro da pasta varnish, crie um ficheiro chamado default.vcl. Isto diz ao Varnish onde encontrar o servidor backend NGINX:

```
vcl 4.1;
backend default {
    .host = "nginx";
    .port = "80";
}
```

- **Dica:** A linha .host = "nginx" usa o nome do serviço do ficheiro compose. O Docker Compose cria automaticamente uma entrada DNS para que os serviços se possam alcançar uns aos outros pelo nome.

4. Crie o Ficheiro Compose:

- Na raiz da sua pasta ex04, crie o compose.yml:

```
services:
  cache:
    image: varnish:stable
    volumes:
      - ./varnish:/etc/varnish:ro
    ports:
      - "8080:80"
    depends_on:
      - nginx
    restart: unless-stopped
  nginx:
    image: nginx:alpine
    volumes:
      - ./my-dynamic-website:/usr/share/nginx/html:ro
    # Sem portas expostas: o nginx só é acessível
    # a partir do interior da rede Docker
```

5. Execute e Verifique:

- Inicie os serviços:
docker compose up -d
- Verifique se ambos os serviços estão a correr:
docker compose ps
Deverá ver dois contentores listados, ambos num estado "running".
- Abra o seu navegador em http://localhost:8080. Deverá ver a sua página web com o GIF. A chave aqui é que o **Varnish** serviu-lhe a página, não o NGINX diretamente.
- **Veja a cache em ação:** Verifique os logs do NGINX para o primeiro pedido:
docker compose logs nginx
- Agora, atualize a página do seu navegador várias vezes. Verifique os logs do nginx novamente:
docker compose logs nginx

Deverá ver **nenhuma nova entrada de log**, porque o Varnish está a servir o conteúdo da sua cache sem contactar o backend NGINX.

- **Bónus – inspecione os cabeçalhos HTTP** para confirmar a cache:

```
curl -I http://localhost:8080
```

Procure cabeçalhos como X-Varnish e Age. Um valor de Age superior a 0 confirma que a resposta foi servida a partir da cache.

Conceito Chave – Service Discovery: O Docker Compose coloca todos os serviços numa rede partilhada por defeito. Os serviços podem alcançar-se uns aos outros usando o seu nome de serviço como hostname (por exemplo, nginx). Apenas os serviços com mapeamentos de portas (ports) são acessíveis a partir da máquina anfitriã.

Limpeza:

```
docker compose down
```

Exercício 5: Implementar uma Aplicação do Mundo Real

Objetivo: Aprender a ler documentação oficial e a implementar um serviço complexo auto-hospedado à sua escolha usando o Docker Compose.

1. A partir do seu diretório de trabalho, crie a Estrutura de Ficheiros:

```
mkdir ex05
cd ex05
```

2. Escolha um Serviço: Vá a [LinuxServer.io](https://lscr.io/linuxserver) e navegue pela lista de imagens populares. Escolha uma que lhe interesse, por exemplo:

- **Jellyfin:** Um servidor multimédia para os seus filmes e música.
- **Nextcloud:** Uma nuvem pessoal para ficheiros, contactos e calendários.
- **Home Assistant:** Uma plataforma de automação residencial de código aberto.

3. Leia a Documentação: Na página da imagem escolhida, encontre a secção “Docker Compose”. Leia-a com atenção, prestando especial atenção a:

- **Volumes (- ./config:/config):** É aqui que a configuração e os dados da aplicação serão armazenados na sua máquina anfitriã. Isto garante que os seus dados persistem mesmo que o contentor seja removido.
- **Variáveis de Ambiente (PUID, PGID, TZ):** Estas são críticas para o funcionamento correto.
 - TZ define o seu fuso horário (p. ex., Europe/Lisbon). Pode encontrar a sua string de fuso horário em [Wikipedia: List of tz database time zones](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones).
 - PUID e PGID garantem que os ficheiros criados pelo contentor têm a propriedade correta no anfitrião. Encontre os seus valores executando `id` no seu terminal. Um valor comum é 1000.
- **Portas:** Anote em que porta a aplicação escuta para saber onde aceder-lhe no seu navegador.

4. Crie o seu `compose.yml`: Com base na documentação, crie o ficheiro. Aqui está um exemplo para o Jellyfin:

```
services:
  jellyfin:
    image: lscr.io/linuxserver/jellyfin:latest
    container_name: jellyfin
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Lisbon
    volumes:
      - ./config:/config
      - ./tvshows:/data/tvshows
      - ./movies:/data/movies
    ports:
```

```
- "8096:8096"
restart: unless-stopped
```

5. Prepare e Implemente:

- Crie as pastas locais que definiu nos seus volumes:

```
mkdir -p config tvshows movies
```

- Inicie a aplicação:

```
docker compose up -d
```

- Monitorize os logs de arranque para verificar a existência de erros:

```
docker compose logs -f
```

Pressione Ctrl+C para parar de seguir os logs (o contentor continua a correr).

6. Explore: Verifique a documentação para o número da porta padrão. Para o Jellyfin, é 8096. Abra o seu navegador em `http://localhost:8096` e siga o assistente de configuração para o seu novo serviço.

Verificação:

- Execute `docker compose ps` para confirmar que o contentor está a correr.
- Execute `docker compose logs` para verificar quaisquer mensagens de erro durante o arranque.
- Se a interface web não carregar, aguarde um minuto – algumas aplicações demoram algum tempo a inicializar no primeiro arranque.

Dica: A política `restart: unless-stopped` significa que o contentor reiniciará automaticamente se falhar ou se o daemon do Docker reiniciar (por exemplo, após uma reinicialização do sistema), a menos que o pare explicitamente com `docker compose down` ou `docker compose stop`.

Limpeza:

```
docker compose down
```

Nota: isto apenas para e remove os contentores. Os seus dados nas pastas de volumes (`config`, `tvshows`, `movies`) são preservados no anfitrião e serão reutilizados se iniciar o serviço novamente.

Referência Rápida: Comandos Essenciais do Docker Compose

A tabela seguinte resume os comandos mais úteis do Docker Compose que irá precisar ao longo destes exercícios e não só.

Comando	Descrição
<code>docker compose up -d</code>	Inicia todos os serviços em modo <i>detached</i> (segundo plano)
<code>docker compose up --build -d</code>	Reconstrói as imagens e inicia todos os serviços
<code>docker compose down</code>	Para e remove todos os contentores e redes
<code>docker compose down -v</code>	O mesmo que o anterior, mas também remove os volumes nomeados
<code>docker compose ps</code>	Lista os serviços a correr e o seu estado
<code>docker compose logs</code>	Mostra os logs de todos os serviços
<code>docker compose logs -f <servico></code>	Segue (tail) os logs para um serviço específico
<code>docker compose exec <servico> sh</code>	Abre uma shell dentro de um contentor a correr
<code>docker compose stop</code>	Para os serviços sem remover os contentores
<code>docker compose start</code>	Inicia serviços previamente parados
<code>docker compose restart</code>	Reinicia todos os serviços
<code>docker compose pull</code>	Puxa (descarrega) as imagens mais recentes para todos os serviços
<code>docker compose config</code>	Valida e mostra o ficheiro <code>compose</code> resolvido

Dicas de Resolução de Problemas

Se encontrar problemas durante os exercícios, tente estes passos:

1. **Porta já em uso:** Se vir um erro como “port is already allocated”, outro serviço (ou um exercício anterior) está a usar essa porta. Pare-o primeiro com `docker compose down` na pasta do outro exercício, ou escolha uma porta de anfitrião diferente (por exemplo, altere “8080:80” para “8081:80”).

2. **Contentor termina imediatamente:** Verifique os logs para compreender o porquê:

```
docker compose logs <nome-do-servico>
```

3. **Alterações nos ficheiros não refletidas:** Se modificou um Dockerfile ou ficheiros copiados com COPY, tem de reconstruir:

```
docker compose up --build -d
```

Se estiver a usar volumes (bind mounts), as alterações devem aparecer imediatamente – tente uma atualização forçada no seu navegador (Ctrl+Shift+R). 4. **Não é possível conectar ao daemon do Docker:** Certifique-se de que o serviço Docker está a correr:

```
sudo systemctl start docker
```

```
sudo systemctl status docker
```

5. **Problemas de espaço em disco:** As imagens e contentores Docker podem acumular-se ao longo do tempo. Limpe os recursos não utilizados com:

```
docker system prune
```

Adicione a flag -a para também remover imagens não utilizadas (não apenas as “dangling”). 6. **Inspeccionar um contentor:** Para explorar o que está a acontecer dentro de um contentor a correr, abra uma shell:

```
docker compose exec <nome-do-servico> sh
```

Isto é útil para verificar o conteúdo de ficheiros, testar conectividade de rede (ping, wget), ou ler logs dentro do contentor.